

Carga e Transformação de Dados

Modelo Analítico de Dados para E-commerce

Sumário

1	Introdução	3
2	Preparação Automatizada do Banco	3
2.1	Identificação do banco de destino	3
2.2	Criação automática do database	4
2.3	Aplicação do modelo físico	4
2.4	Reconstrução controlada	4
3	Arquitetura do ELT	5
3.1	Camada RAW	5
3.2	Camada CORE	5
3.3	Atomicidade do pipeline completo	6
4	Contrato de Ingestão RAW	6
5	Mapeamento RAW para CORE	7
6	Regras de Qualidade e Saneamento	7
7	Implementação	8
8	Configuração e Execução	9
9	Validação e Testes	10
10	Resultado da Carga	11
11	Rastreabilidade e Limites	11
12	Conclusão	12

1 Introdução

Este documento consolida a preparação automatizada do banco de dados, o processo de carga e a transformação do projeto *Modelo Analítico de Dados para E-commerce*, desde a criação da estrutura física no PostgreSQL até a publicação das entidades relacionais no schema *core*.

A preparação da infraestrutura de banco é tratada separadamente do ELT. O componente *database* garante que o banco de destino exista, aplica os scripts versionados do modelo físico, cria os índices e valida a arquitetura. Somente após essa etapa o pipeline ELT realiza a carga dos nove arquivos CSV do *Brazilian E-Commerce Public Dataset by Olist* na RAW e a transformação para o CORE.

A solução busca garantir reprodutibilidade, rastreabilidade, integridade, atomicidade e reconciliação entre fonte e destino.

PostgreSQL → database.setup → RAW → CORE → ANALYTICS → Power BI / SQL

As decisões são descritas por sua finalidade técnica e semântica, sem depender de identificadores internos utilizados no gerenciamento do projeto.

2 Preparação Automatizada do Banco

A criação e a preparação da estrutura física são responsabilidades anteriores ao ELT e, por esse motivo, permanecem isoladas no pacote *database*. Os scripts SQL em *models/physical/* continuam sendo a fonte da verdade para o modelo físico; o código Python atua apenas como orquestrador.

2.1 Identificação do banco de destino

A conexão é definida pela variável de ambiente *DATABASE_URL*. O nome do banco é extraído da própria URL e registrado no log antes de qualquer alteração estrutural.

No ambiente local, a configuração pode ser mantida em arquivo *.env*, ignorado pelo Git:

```
DATABASE_URL=postgresql://usuario:senha@localhost:5432/ecommerce_analytics
```

O arquivo *.env.example* documenta a variável necessária sem expor credenciais reais.

2.2 Criação automática do database

Antes de aplicar a estrutura física, o processo consulta o catálogo `pg_database` por meio do banco administrativo `postgres`. Se o database indicado em `DATABASE_URL` ainda não existir, ele é criado automaticamente.

Essa operação depende de o usuário PostgreSQL possuir permissão para `CREATE DATABASE`. Caso a permissão não esteja disponível, a execução é interrompida e o erro é informado sem tentativa de contorno.

A verificação por catálogo evita dependência de mensagens textuais do servidor e, portanto, não depende do idioma configurado no PostgreSQL.

2.3 Aplicação do modelo físico

Após garantir a existência do database, o processo aplica os artefatos físicos versionados nesta ordem:

1. `models/physical/create_schema.sql`;
2. `models/physical/create_indexes.sql`;
3. `models/physical/validate_schema.sql`.

A validação final confirma a presença dos schemas, tabelas, colunas, identities, PKs, FKs, constraints e índices definidos para a arquitetura física.

2.4 Reconstrução controlada

Quando o banco já existe, a opção `-reset` executa `drop_schema.sql` antes da recriação da estrutura:

```
uv run python -m database.setup --reset
```

O `-reset` não remove o database PostgreSQL. Apenas a estrutura pertencente ao projeto é reconstruída. Se o database for criado na própria execução, a etapa de remoção é omitida por não existir estrutura anterior a ser descartada.

Para um ambiente novo, a preparação pode ser executada simplesmente por:

```
uv run python -m database.setup
```

3 Arquitetura do ELT

3.1 Camada RAW

A camada RAW preserva os valores recebidos da fonte. Existe uma tabela correspondente a cada CSV; as colunas de negócio mantêm os nomes originais e são armazenadas inicialmente como `text`.

Cada ocorrência recebe os metadados técnicos:

- `_id_raw`: identificador técnico da ocorrência carregada;
- `_arquivo_origem`: identificação do arquivo de origem;
- `_carregado_em`: instante em que o registro foi incorporado à RAW.

A política de recarga é substitutiva e atômica: uma nova carga aprovada substitui integralmente a anterior. Falhas durante a operação provocam *rollback*, preservando o estado previamente publicado.

3.2 Camada CORE

O CORE materializa o modelo relacional aprovado. As transformações incluem:

- conversões explícitas de tipos;
- renomeações de atributos;
- aplicação das constraints físicas;
- derivação de prefixos de CEP;
- consolidação das duplicidades integrais de geolocalização;
- reconciliação dos volumes inseridos.

A ordem de dependências adotada é:

1. `core.prefixo_cep`;
2. `core.cliente`, `core.produto` e `core.vendedor`;
3. `core.pedido` e `core.geolocalizacao`;
4. `core.item_pedido`, `core.pagamento` e `core.avaliacao`.

3.3 Atomicidade do pipeline completo

O comando completo usa uma única conexão PostgreSQL para o fluxo CSV → RAW → CORE. O controle transacional é realizado pelo `psycopg`. Se qualquer arquivo, conversão, constraint ou reconciliação falhar, RAW e CORE retornam conjuntamente ao estado anterior.

4 Contrato de Ingestão RAW

A ingestão contempla exclusivamente os nove arquivos contratados. Arquivos adicionais exigem alteração explícita do contrato e dos artefatos físicos.

Arquivo	Linhas	Destino RAW
olist_customers_dataset.csv	99.441	olist_customers
olist_geolocation_dataset.csv	1.000.163	olist_geolocation
olist_order_items_dataset.csv	112.650	olist_order_items
olist_order_payments_dataset.csv	103.886	olist_order_payments
olist_order_reviews_dataset.csv	99.224	olist_order_reviews
olist_orders_dataset.csv	99.441	olist_orders
olist_products_dataset.csv	32.951	olist_products
olist_sellers_dataset.csv	3.095	olist_sellers
product_category_name_translation.csv	71	product_category_name_translation
Total	1.550.922	9 tabelas

A pré-validação confirma, antes de qualquer alteração no banco:

- presença dos nove arquivos obrigatórios;
- leitura UTF-8 ou UTF-8 com BOM;
- cabeçalho exato e na ordem contratada;
- quantidade correta de campos em todas as linhas;
- sintaxe CSV válida;
- quantidade de registros;
- tamanho e hash SHA-256 de cada fonte.

Campos vazios podem ser carregados como SQL NULL. Strings não vazias semelhantes a marcadores de ausência não são reinterpretadas. Duplicidades e inconsistências de negócio permanecem preservadas na RAW.

5 Mapeamento RAW para CORE

O mapeamento preserva identificadores de 32 caracteres como texto, prefixos de CEP como cinco caracteres, valores monetários em `numeric(12,2)` e datas em `timestamp without time zone`. Valores inválidos não são corrigidos ou descartados silenciosamente.

Origem	Destino CORE	Tratamento principal
<code>olist_customers</code>	<code>cliente</code>	Identificadores, CEP, cidade e UF
<code>olist_orders</code>	<code>pedido</code>	Status e eventos temporais
<code>olist_order_items</code>	<code>item_pedido</code>	Item, produto, vendedor, preço e frete
<code>olist_products</code>	<code>produto</code>	Categoria, descritores e medidas físicas
<code>olist_sellers</code>	<code>vendedor</code>	Identificador, CEP, cidade e UF
<code>olist_order_payments</code>	<code>pagamento</code>	Sequência, tipo, parcelas e valor
<code>olist_order_reviews</code>	<code>avaliacao</code>	PK composta, nota e datas
Clientes + vendedores + geolocalização	<code>prefixo_cep</code>	União distinta dos prefixos
<code>olist_geolocation</code>	<code>geolocalizacao</code>	Deduplicação exata e identity

Título e mensagem das avaliações e a tabela de tradução de categorias permanecem disponíveis na RAW porque não possuem destino no CORE aprovado.

6 Regras de Qualidade e Saneamento

As regras são classificadas como bloqueantes, transformações ou informativas. Violações bloqueantes impedem a publicação do CORE; transformações aprovadas alteram o dado de forma determinística e reconciliável; características válidas incomuns são preservadas.

Entre os controles aplicados estão:

- preservação de NOT NULL, PK, FK, UNIQUE e CHECK;

- validação do formato dos identificadores quando aplicável;
- prefixos de CEP com cinco dígitos e preservação de zeros à esquerda;
- validação dos domínios de UF, status de pedido e tipo de pagamento;
- conversão estrita de datas para timestamp sem fuso;
- valores monetários não negativos com precisão decimal exata;
- notas de avaliação entre 1 e 5;
- latitude e longitude dentro dos limites globais;
- integridade referencial entre as entidades carregadas.

O profiling de qualidade avaliou 113 regras sobre o snapshot e encontrou **zero violações bloqueantes**. Foram identificadas 261.831 duplicidades integrais de geo-localização, consolidadas de forma determinística, e 19.177 prefixos de CEP distintos derivados.

7 Implementação

Artefato	Responsabilidade
models/physical/*.sql	Fonte da verdade da estrutura física, índices, desmontagem e validação
database/setup.py	Criação do database quando necessário e orquestração da preparação física
elt/raw_loader.py	Pré-validação, COPY, recarga substitutiva e reconciliação RAW
elt/sql/load_core.sql	Transformações SQL na ordem de dependências
elt/core_loader.py	Execução e reconciliação do CORE
elt/pipeline.py	Orquestração atômica CSV → RAW → CORE
config/raw_load.toml	Contrato operacional das fontes RAW
config/core_load.toml	Configuração da transformação CORE
tests/test_database_setup.py	Testes automatizados da preparação do banco
tests/	Testes automatizados dos loaders, pipeline e componentes auxiliares

8 Configuração e Execução

Credenciais não são versionadas. No ambiente local, a conexão pode ser armazenada em `.env`, ignorado pelo Git, enquanto um arquivo de exemplo sem segredo documenta a variável necessária.

```
DATABASE_URL=postgresql://usuario:senha@localhost:5432/ecommerce_analytics
```

A preparação inicial do banco é executada com:

```
uv run python -m database.setup
```

Se o database ainda não existir, ele é criado automaticamente antes da aplicação do modelo físico. Para reconstruir a estrutura de um database já existente:

```
uv run python -m database.setup --reset
```

Após a preparação física, a execução operacional do pipeline completo é:

```
uv run python -m elt.pipeline
```

O fluxo reproduzível completo é, portanto:

```
uv run python -m database.setup --reset
uv run python -m elt.pipeline
```

A pré-validação isolada continua disponível por:

```
uv run python -m elt.pipeline --validate-only
```

Cada execução completa gera um log JSON local em:

```
outputs/data-loading/logs/full_load_<run_id>.json
```

O registro contém perfis e hashes das fontes, reconciliações RAW e CORE, hash do SQL de transformação, horários, modo, status e erro quando houver. A URL de conexão não é registrada.

9 Validação e Testes

A implementação do ELT foi validada em PostgreSQL 18.6 temporário e isolado, com duas execuções completas consecutivas sem acumulação. Um erro deliberado ao final da transformação confirmou rollback conjunto de RAW e CORE.

A execução em base persistente também foi concluída com status **APPROVED**, após recriação da estrutura física e dos índices.

A automação de preparação do banco foi validada manualmente em dois cenários: uso de database já existente e criação automática de um database inexistente. Neste segundo cenário, o processo identificou a ausência pelo catálogo `pg_database`, criou o database, aplicou a estrutura, criou os índices e concluiu a validação física com sucesso.

A suíte específica de `database.setup` contém oito testes automatizados aprovados, cobrindo:

- extração do nome do database a partir da URL;
- construção da URL administrativa apontando para `postgres`;
- não criação quando o database já existe;
- criação quando o database não existe;
- execução normal sem remoção de schema;
- ordem correta dos scripts no modo `-reset`;
- omissão do `drop_schema.sql` quando o database acaba de ser criado;
- falha controlada quando `DATABASE_URL` não está configurada.

Os testes previamente existentes continuam exercitando os loaders e o pipeline, incluindo validação sem conexão, ausência de `DATABASE_URL`, compartilhamento da mesma conexão entre RAW e CORE, propagação de falha no CORE e presença das reconciliações RAW e CORE no resultado.

A suíte completa é executada com:

```
uv run python -m unittest discover -s tests -v
```

10 Resultado da Carga

Os volumes reconciliados no CORE para o snapshot validado são:

Tabela CORE	Registros reconciliados
prefixo_cep	19.177
cliente	99.441
produto	32.951
vendedor	3.095
pedido	99.441
item_pedido	112.650
pagamento	103.886
avaliacao	99.224
geolocalizacao	738.332
Total físico	1.308.197

O total CORE não deve ser comparado diretamente ao total RAW como se ambas as camadas possuíssem a mesma granularidade. A diferença decorre da entidade derivada de prefixos, da consolidação de geolocalização, da tabela de tradução sem destino e das granularidades próprias das entidades.

11 Rastreabilidade e Limites

A RAW mantém o valor original e os metadados técnicos necessários para localizar a ocorrência carregada. Falhas do PostgreSQL e divergências de reconciliação interrompem a transação e são registradas no log operacional.

A preparação do banco registra explicitamente o database de destino antes das operações estruturais. O processo não armazena a URL de conexão nos logs e não remove o database PostgreSQL no modo `-reset`; apenas os schemas e objetos pertencentes ao modelo são reconstruídos.

O contrato de qualidade prevê que violações bloqueantes sejam associadas, quando aplicável, à carga, tabela, arquivo, `_id_raw`, coluna, valor original, regra e motivo. A implementação atual garante bloqueio transacional e registro da falha, mas o detalhamento estruturado por regra permanece um ponto de evolução para cenários em que o snapshot apresente violações bloqueantes.

Agendamento, observabilidade de produção e objetos do schema `analytics` não

fazem parte desta entrega.

12 Conclusão

O processo está implementado de forma reproduzível desde a preparação do banco até a publicação das entidades no CORE. O componente `database.setup` garante que o database exista, aplica e valida o modelo físico sem misturar essa responsabilidade ao ELT. Em seguida, os nove CSVs são pré-validados, carregados e reconciliados na RAW, enquanto as transformações materializam as nove tabelas do CORE respeitando dependências e constraints.

A automação reduz etapas manuais, elimina a necessidade de executar individualmente os scripts físicos por uma interface gráfica e torna a reconstrução do ambiente verificável por comandos versionados.

Com a preparação física e a execução integrada aprovadas, o banco fica preparado para a validação final de reconciliação e, posteriormente, para a construção das estruturas analíticas destinadas ao consumo.